

Ajmal_Assignment5

November 24, 2025

Part 1- Video read/write

```
[5]: from google.colab.patches import cv2_imshow
import cv2
import numpy as np
```

```
[9]: cap = cv2.VideoCapture('/content/Bangkok.mp4')
video_fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))

print(f"Frame Per second: {video_fps } \nTotal Frames: {total_frames} \nHeight: {height} \nWidth: {width}")
```

Frame Per second: 25.0

Total Frames: 650

Height: 720

Width: 1280

```
[11]: #b) Play the file- method 1
# Check if camera opened successfully
if (cap.isOpened() == False):
    print("Error opening video stream or file")

frame_count = 0
max_frames_to_show = 5 # Display a few frames to demonstrate

while(True):
    ret, frame = cap.read()
    if ret == True:
        # Display the resulting frame
        cv2_imshow(frame)
        frame_count += 1
        if frame_count >= max_frames_to_show:
            break
    # Break the loop
else:
```

`break`

When everything done, release the video capture object

`cap.release()`

In Colab, `cv2.destroyAllWindows()` is not needed/functional with `cv2.imshow`







```
[14]: #Play the file- method 2
from IPython.display import HTML
from base64 import b64encode
video_path = '/content/Bangkok.mp4'

mp4 = open(video_path, 'rb').read()
decoded_vid = "data:video/mp4;base64," + b64encode(mp4).decode()
HTML(f'<video width=400 controls><source src={decoded_vid} type="video/mp4"></>video>')
```

```
[14]: <IPython.core.display.HTML object>
```

```
[17]: # we are using x264 codec for mp4
out_video_path = '/content/filtered_video.mp4'
fourcc = cv2.VideoWriter_fourcc(*'X264')
writer = cv2.VideoWriter(out_video_path, apiPreference=0, fourcc=fourcc,
                        fps=video_fps, frameSize=(int(width), int(height)))
```

```
[21]: # Creating a VideoCapture object to read the video
video_path = '/content/Bangkok.mp4'
cap = cv2.VideoCapture(video_path)

frame_count = 0
max_frames_to_show = 3 # We only want to print 3 frames

# Loop until the end of the video or max_frames_to_show is reached
```



```

while cap.isOpened() and frame_count < max_frames_to_show:

    # Capture frame-by-frame
    ret, frame = cap.read()

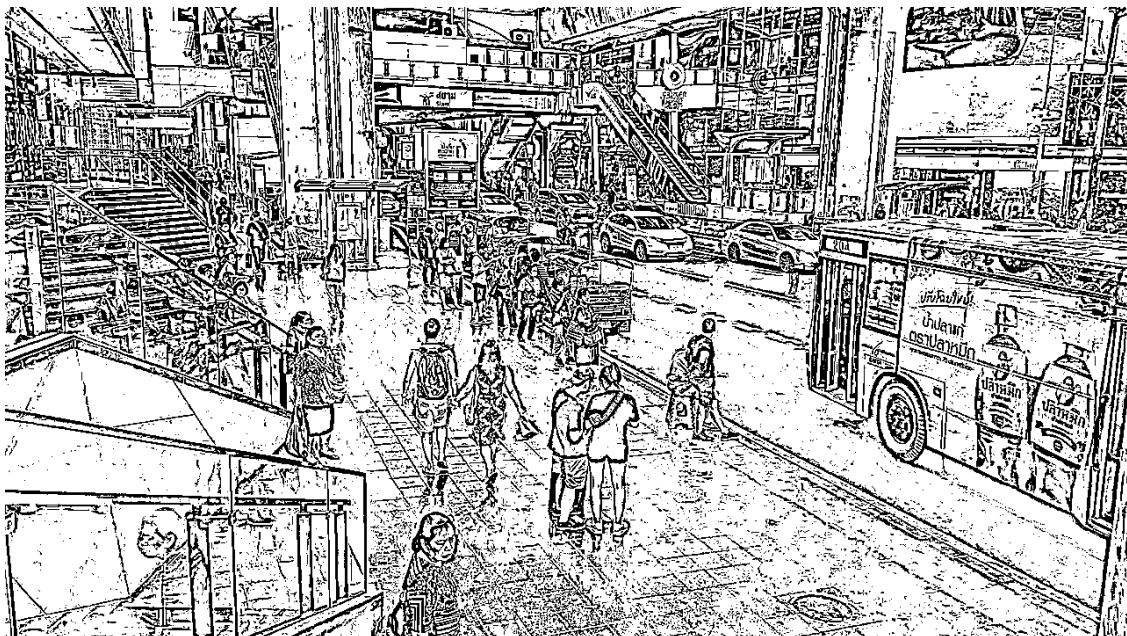
    # Display the resulting frame
    if ret==True:
        # conversion of BGR to grayscale is necessary to apply this operation
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

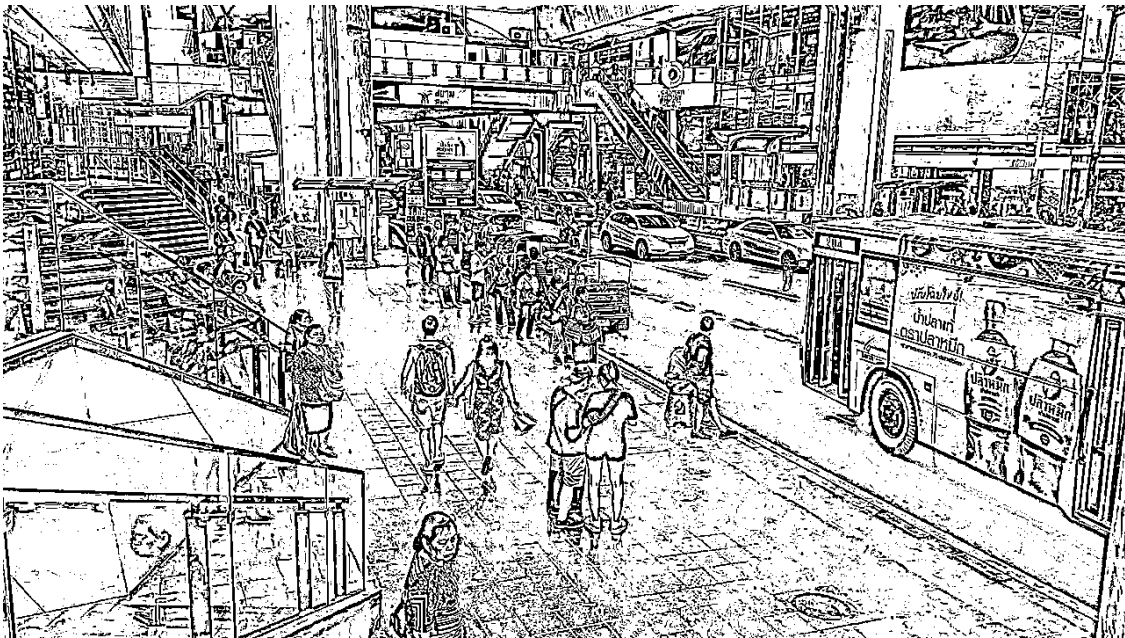
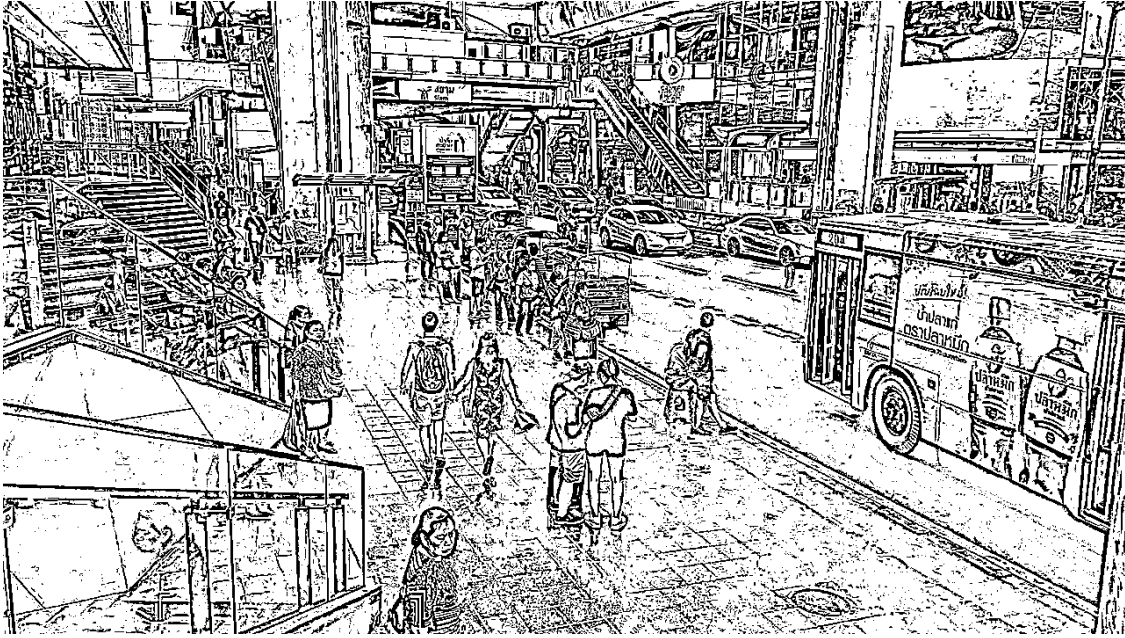
        # adaptive thresholding to use different threshold
        # values on different regions of the frame.
        Thresh = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
↪cv2.THRESH_BINARY, 11, 2)

        cv2_imshow(Thresh)
        frame_count += 1
    else:
        break

# release the video capture object
cap.release()
# Closes all the windows currently opened. Not needed with cv2_imshow in Colab.
# cv2.destroyAllWindows()

```





Part 2- Pose estimation in video

```
[19]: import tensorflow as tf
import tensorflow_hub as hub
#from tensorflow_docs.vis import embed
import numpy as np
```

```

import cv2

# Import matplotlib libraries
from matplotlib import pyplot as plt
from matplotlib.collections import LineCollection
import matplotlib.patches as patches

# Some modules to display an animation using imageio.
# Some modules to display an animation using imageio.
import imageio
from IPython.display import HTML, display

from IPython.display import Image as IPythonImage
from PIL import Image as PILImage
import io

```

```

[22]: model_name = "movenet_lightning"

if "tflite" in model_name:
    if "movenet_lightning_f16" in model_name:
        !wget -q -O model.tflite https://tfhub.dev/google/lite-model/movenet/
        ↪singlepose/lightning/tflite/float16/4?lite-format=tflite
        input_size = 192
    elif "movenet_thunder_f16" in model_name:
        !wget -q -O model.tflite https://tfhub.dev/google/lite-model/movenet/
        ↪singlepose/thunder/tflite/float16/4?lite-format=tflite
        input_size = 256
    elif "movenet_lightning_int8" in model_name:
        !wget -q -O model.tflite https://tfhub.dev/google/lite-model/movenet/
        ↪singlepose/lightning/tflite/int8/4?lite-format=tflite
        input_size = 192
    elif "movenet_thunder_int8" in model_name:
        !wget -q -O model.tflite https://tfhub.dev/google/lite-model/movenet/
        ↪singlepose/thunder/tflite/int8/4?lite-format=tflite
        input_size = 256
    else:
        raise ValueError("Unsupported model name: %s" % model_name)

# Initialize the TFLite interpreter
interpreter = tf.lite.Interpreter(model_path="model.tflite")
interpreter.allocate_tensors()

def movenet(input_image):
    """Runs detection on an input image.

    Args:
        input_image: A [1, height, width, 3] tensor represents the input image

```

pixels. Note that the height/width should already be resized and match the expected input resolution of the model before passing into this function.

Returns:

A [1, 1, 17, 3] float numpy array representing the predicted keypoint coordinates and scores.

"""

TF Lite format expects tensor type of uint8.

input_image = tf.cast(input_image, dtype=tf.uint8)

input_details = interpreter.get_input_details()

output_details = interpreter.get_output_details()

interpreter.set_tensor(input_details[0]['index'], input_image.numpy())

Invoke inference.

interpreter.invoke()

Get the model prediction.

keypoints_with_scores = interpreter.get_tensor(output_details[0]['index'])

return keypoints_with_scores

else:

if "movenet_lightning" in model_name:

module = hub.load("https://tfhub.dev/google/movenet/singlepose/lightning/4")

input_size = 192

elif "movenet_thunder" in model_name:

module = hub.load("https://tfhub.dev/google/movenet/singlepose/thunder/4")

input_size = 256

else:

raise ValueError("Unsupported model name: %s" % model_name)

def movenet(input_image):

"""Runs detection on an input image.

Args:

input_image: A [1, height, width, 3] tensor represents the input image pixels. Note that the height/width should already be resized and match

the

expected input resolution of the model before passing into this

function.

Returns:

A [1, 1, 17, 3] float numpy array representing the predicted keypoint coordinates and scores.

"""

model = module.signatures['serving_default']

SavedModel format expects tensor type of int32.


```

input_image = tf.cast(input_image, dtype=tf.int32)
# Run model inference.
outputs = model(input_image)
# Output is a [1, 1, 17, 3] tensor.
keypoints_with_scores = outputs['output_0'].numpy()
return keypoints_with_scores

```

```

[23]: !wget -q -O dance.gif https://github.com/tensorflow/tfjs-models/raw/master/
      ↪pose-detection/assets/dance_input.gif

```

```

[24]: # Load the input image.
image_path = 'dance.gif'
image = tf.io.read_file(image_path)
image = tf.image.decode_gif(image)

```

```

[57]: #All the required code are put in this cell you don't need to modify anything_
      ↪in this cell
# Confidence score to determine whether a keypoint prediction is reliable.
MIN_CROP_KEYPOINT_SCORE = 0.2

KEYPOINT_DICT = {
    'nose': 0,
    'left_eye': 1,
    'right_eye': 2,
    'left_ear': 3,
    'right_ear': 4,
    'left_shoulder': 5,
    'right_shoulder': 6,
    'left_elbow': 7,
    'right_elbow': 8,
    'left_wrist': 9,
    'right_wrist': 10,
    'left_hip': 11,
    'right_hip': 12,
    'left_knee': 13,
    'right_knee': 14,
    'left_ankle': 15,
    'right_ankle': 16
}

KEYPOINT_EDGE_INDS_TO_COLOR = {
    (0, 1): 'm',
    (0, 2): 'c',
    (1, 3): 'm',
    (2, 4): 'c',
    (0, 5): 'm',
    (0, 6): 'c',

```

```

(5, 7): 'm',
(7, 9): 'm',
(6, 8): 'c',
(8, 10): 'c',
(5, 6): 'y',
(5, 11): 'm',
(6, 12): 'c',
(11, 12): 'y',
(11, 13): 'm',
(13, 15): 'm',
(12, 14): 'c',
(14, 16): 'c'
}

def init_crop_region(image_height, image_width):
    """Defines the default crop region.

    The function provides the initial crop region (pads the full image from both
    sides to make it a square image) when the algorithm cannot reliably determine
    the crop region from the previous frame.
    """

    if image_width > image_height:
        box_height = image_width / image_height
        box_width = 1.0
        y_min = (image_height / 2 - image_width / 2) / image_height
        x_min = 0.0
    else:
        box_height = 1.0
        box_width = image_height / image_width
        y_min = 0.0
        x_min = (image_width / 2 - image_height / 2) / image_width

    return {
        'y_min': y_min,
        'x_min': x_min,
        'y_max': y_min + box_height,
        'x_max': x_min + box_width,
        'height': box_height,
        'width': box_width
    }

def torso_visible(keypoints):
    """Checks whether there are enough torso keypoints.

    This function checks whether the model is confident at predicting one of the
    shoulders/hips which is required to determine a good crop region.
    """

    return ((keypoints[0, 0, KEYPOINT_DICT['left_hip'], 2] >

```

```

        MIN_CROP_KEYPOINT_SCORE or
        keypoints[0, 0, KEYPOINT_DICT['right_hip'], 2] >
        MIN_CROP_KEYPOINT_SCORE) and
        (keypoints[0, 0, KEYPOINT_DICT['left_shoulder'], 2] >
        MIN_CROP_KEYPOINT_SCORE or
        keypoints[0, 0, KEYPOINT_DICT['right_shoulder'], 2] >
        MIN_CROP_KEYPOINT_SCORE))

def _keypoints_and_edges_for_display(keypoints_with_scores,
                                     height,
                                     width,
                                     keypoint_threshold=0.11):
    """Returns high confidence keypoints and edges for visualization.

    Args:
        keypoints_with_scores: A numpy array with shape [1, 1, 17, 3] representing
            the keypoint coordinates and scores returned from the MoveNet model.
        height: height of the image in pixels.
        width: width of the image in pixels.
        keypoint_threshold: minimum confidence score for a keypoint to be
            visualized.

    Returns:
        A (keypoints_xy, edges_xy, edge_colors) containing:
        * the coordinates of all keypoints of all detected entities;
        * the coordinates of all skeleton edges of all detected entities;
        * the colors in which the edges should be plotted.
    """

    keypoints_all = []
    keypoint_edges_all = []
    edge_colors = []
    num_instances, _, _, _ = keypoints_with_scores.shape
    for idx in range(num_instances):
        kpts_x = keypoints_with_scores[0, idx, :, 1]
        kpts_y = keypoints_with_scores[0, idx, :, 0]
        kpts_scores = keypoints_with_scores[0, idx, :, 2]
        kpts_absolute_xy = np.stack(
            [width * np.array(kpts_x), height * np.array(kpts_y)], axis=-1)
        kpts_above_thresh_absolute = kpts_absolute_xy[
            kpts_scores > keypoint_threshold, :]
        keypoints_all.append(kpts_above_thresh_absolute)

    for edge_pair, color in KEYPOINT_EDGE_INDS_TO_COLOR.items():
        if (kpts_scores[edge_pair[0]] > keypoint_threshold and
            kpts_scores[edge_pair[1]] > keypoint_threshold):
            x_start = kpts_absolute_xy[edge_pair[0], 0]

```



```

        y_start = kpts_absolute_xy[edge_pair[0], 1]
        x_end = kpts_absolute_xy[edge_pair[1], 0]
        y_end = kpts_absolute_xy[edge_pair[1], 1]
        line_seg = np.array([[x_start, y_start], [x_end, y_end]])
        keypoint_edges_all.append(line_seg)
        edge_colors.append(color)
    if keypoints_all:
        keypoints_xy = np.concatenate(keypoints_all, axis=0)
    else:
        keypoints_xy = np.zeros((0, 17, 2))

    if keypoint_edges_all:
        edges_xy = np.stack(keypoint_edges_all, axis=0)
    else:
        edges_xy = np.zeros((0, 2, 2))
    return keypoints_xy, edges_xy, edge_colors

def progress(value, max=100):
    return HTML("""
        <progress
            value='{value}'
            max='{max}',
            style='width: 100%'
        >
            {value}
        </progress>
    """).format(value=value, max=max)

def crop_and_resize(image, crop_region, crop_size):
    """Crops and resize the image to prepare for the model input."""
    boxes=[[crop_region['y_min'], crop_region['x_min'],
            crop_region['y_max'], crop_region['x_max']]]
    output_image = tf.image.crop_and_resize(
        image, box_indices=[0], boxes=boxes, crop_size=crop_size)
    return output_image

def run_inference(movenet, image, crop_region, crop_size):
    """Runs model inference on the cropped region.

    The function runs the model inference on the cropped region and updates the
    model output to the original image coordinate system.
    """
    image_height, image_width, _ = image.shape
    input_image = crop_and_resize(
        tf.expand_dims(image, axis=0), crop_region, crop_size=crop_size)
    # Run model inference.

```

```

keypoints_with_scores = movenet(input_image)
# Update the coordinates.
for idx in range(17):
    keypoints_with_scores[0, 0, idx, 0] = (
        crop_region['y_min'] * image_height +
        crop_region['height'] * image_height *
        keypoints_with_scores[0, 0, idx, 0]) / image_height
    keypoints_with_scores[0, 0, idx, 1] = (
        crop_region['x_min'] * image_width +
        crop_region['width'] * image_width *
        keypoints_with_scores[0, 0, idx, 1]) / image_width
return keypoints_with_scores

def draw_prediction_on_image(
    image, keypoints_with_scores, crop_region=None, close_figure=False,
    output_image_height=None):
    """Draws the keypoint predictions on image.

    Args:
        image: A numpy array with shape [height, width, channel] representing the
            pixel values of the input image.
        keypoints_with_scores: A numpy array with shape [1, 1, 17, 3] representing
            the keypoint coordinates and scores returned from the MoveNet model.
        crop_region: A dictionary that defines the coordinates of the bounding box
            of the crop region in normalized coordinates (see the init_crop_region
            function below for more detail). If provided, this function will also
            draw the bounding box on the image.
        output_image_height: An integer indicating the height of the output image.
            Note that the image aspect ratio will be the same as the input image.

    Returns:
        A numpy array with shape [out_height, out_width, channel] representing the
        image overlaid with keypoint predictions.
    """
    height, width, channel = image.shape
    aspect_ratio = float(width) / height
    fig, ax = plt.subplots(figsize=(12 * aspect_ratio, 12))
    # To remove the huge white borders
    fig.tight_layout(pad=0)
    ax.margins(0)
    ax.set_yticklabels([])
    ax.set_xticklabels([])
    plt.axis('off')

    im = ax.imshow(image)
    line_segments = LineCollection([], linewidths=(4), linestyle='solid')
    ax.add_collection(line_segments)

```

```

# Turn off tick labels
scat = ax.scatter([], [], s=60, color='#FF1493', zorder=3)

(keypoint_locs, keypoint_edges,
 edge_colors) = _keypoints_and_edges_for_display(
    keypoints_with_scores, height, width)

line_segments.set_segments(keypoint_edges)
line_segments.set_color(edge_colors)
if keypoint_edges.shape[0]:
    line_segments.set_segments(keypoint_edges)
    line_segments.set_color(edge_colors)
if keypoint_locs.shape[0]:
    scat.set_offsets(keypoint_locs)

if crop_region is not None:
    xmin = max(crop_region['x_min'] * width, 0.0)
    ymin = max(crop_region['y_min'] * height, 0.0)
    rec_width = min(crop_region['x_max'], 0.99) * width - xmin
    rec_height = min(crop_region['y_max'], 0.99) * height - ymin
    rect = patches.Rectangle(
        (xmin,ymin),rec_width,rec_height,
        linewidth=1,edgecolor='b',facecolor='none')
    ax.add_patch(rect)

# Modified to use BytesIO for image conversion
buf = io.BytesIO()
fig.canvas.print_png(buf)
buf.seek(0)
image_from_plot = PILImage.open(buf).convert('RGB')
image_from_plot = np.array(image_from_plot)

plt.close(fig)
if output_image_height is not None:
    output_image_width = int(output_image_height / height * width)
    image_from_plot = cv2.resize(
        image_from_plot, dsize=(output_image_width, output_image_height),
        interpolation=cv2.INTER_CUBIC)
return image_from_plot

def determine_crop_region(
    keypoints, image_height,
    image_width):
    """Determines the region to crop the image for the model to run inference on.

    The algorithm uses the detected joints from the previous frame to estimate
    the square region that encloses the full body of the target person and

```

centers at the midpoint of two hip joints. The crop size is determined by the distances between each joints and the center point. When the model is not confident with the four torso joint predictions, the function returns a default crop which is the full image padded to square.

```
target_keypoints = {}
for joint in KEYPOINT_DICT.keys():
    target_keypoints[joint] = [
        keypoints[0, 0, KEYPOINT_DICT[joint], 0] * image_height,
        keypoints[0, 0, KEYPOINT_DICT[joint], 1] * image_width
    ]

if torso_visible(keypoints):
    center_y = (target_keypoints['left_hip'][0] +
                target_keypoints['right_hip'][0]) / 2;
    center_x = (target_keypoints['left_hip'][1] +
                target_keypoints['right_hip'][1]) / 2;

    (max_torso_yrange, max_torso_xrange,
     max_body_yrange, max_body_xrange) = determine_torso_and_body_range(
        keypoints, target_keypoints, center_y, center_x)

    crop_length_half = np.amax(
        [max_torso_xrange * 1.9, max_torso_yrange * 1.9,
         max_body_yrange * 1.2, max_body_xrange * 1.2])

    tmp = np.array(
        [center_x, image_width - center_x, center_y, image_height - center_y])
    crop_length_half = np.amin(
        [crop_length_half, np.amax(tmp)]);

    crop_corner = [center_y - crop_length_half, center_x - crop_length_half];

    if crop_length_half > max(image_width, image_height) / 2:
        return init_crop_region(image_height, image_width)
    else:
        crop_length = crop_length_half * 2;
        return {
            'y_min': crop_corner[0] / image_height,
            'x_min': crop_corner[1] / image_width,
            'y_max': (crop_corner[0] + crop_length) / image_height,
            'x_max': (crop_corner[1] + crop_length) / image_width,
            'height': (crop_corner[0] + crop_length) / image_height -
                crop_corner[0] / image_height,
            'width': (crop_corner[1] + crop_length) / image_width -
                crop_corner[1] / image_width
        }
}
```



```

else:
    return init_crop_region(image_height, image_width)

def determine_torso_and_body_range(
    keypoints, target_keypoints, center_y, center_x):
    """Calculates the maximum distance from each keypoints to the center location.

    The function returns the maximum distances from the two sets of keypoints:
    full 17 keypoints and 4 torso keypoints. The returned information will be
    used to determine the crop size. See determineCropRegion for more detail.
    """

    torso_joints = ['left_shoulder', 'right_shoulder', 'left_hip', 'right_hip']
    max_torso_yrange = 0.0
    max_torso_xrange = 0.0
    for joint in torso_joints:
        dist_y = abs(center_y - target_keypoints[joint][0])
        dist_x = abs(center_x - target_keypoints[joint][1])
        if dist_y > max_torso_yrange:
            max_torso_yrange = dist_y
        if dist_x > max_torso_xrange:
            max_torso_xrange = dist_x

    max_body_yrange = 0.0
    max_body_xrange = 0.0
    for joint in KEYPOINT_DICT.keys():
        if keypoints[0, 0, KEYPOINT_DICT[joint], 2] < MIN_CROP_KEYPOINT_SCORE:
            continue
        dist_y = abs(center_y - target_keypoints[joint][0]);
        dist_x = abs(center_x - target_keypoints[joint][1]);
        if dist_y > max_body_yrange:
            max_body_yrange = dist_y

        if dist_x > max_body_xrange:
            max_body_xrange = dist_x

    return [max_torso_yrange, max_torso_xrange, max_body_yrange, max_body_xrange]

def to_gif(images, fps):
    """Converts image sequence (4D numpy array) to gif."""
    imageio.mimsave('./animation.gif', images, fps=fps)

def progress(value, max=100):
    return HTML("""
        <progress
            value='{value}'
            max='{max}',

```

```

        style='width: 100%'
    >
        {value}
    </progress>
    """.format(value=value, max=max))

```

```

[58]: # Load the input image.
num_frames, image_height, image_width, _ = image.shape
crop_region = init_crop_region(image_height, image_width)

output_images = []
bar = display(progress(0, num_frames-1), display_id=True)
for frame_idx in range(num_frames):
    keypoints_with_scores = run_inference(
        movenet, image[frame_idx, :, :, :], crop_region,
        crop_size=[input_size, input_size])
    output_images.append(draw_prediction_on_image(
        image[frame_idx, :, :, :].numpy().astype(np.int32),
        keypoints_with_scores, crop_region=None,
        close_figure=True, output_image_height=300))
    crop_region = determine_crop_region(
        keypoints_with_scores, image_height, image_width)
    bar.update(progress(frame_idx, num_frames-1))

# Prepare gif visualization.
output = np.stack(output_images, axis=0)
to_gif(output, fps=10) # Assuming a default FPS for the GIF animation
IPythonImage(filename='./animation.gif')

```

<IPython.core.display.HTML object>

[58]: <IPython.core.display.Image object>

Part 3- Using webcam video

```

[ ]: # Step 1: Install necessary libraries
!pip install torch torchvision torchaudio
!pip install ultralytics
!pip install ipywidgets

```

```

[27]: # Load Required libraries
import cv2
import numpy as np
import torch
import requests
import ipywidgets as widgets
from IPython.display import display, Javascript
from google.colab.output import eval_js

```

```
import base64
from io import BytesIO
from PIL import Image
from ultralytics import YOLO
```

Creating new Ultralytics Settings v0.0.6 file
 View Ultralytics Settings with 'yolo settings' or at
 '/root/.config/Ultralytics/settings.json'
 Update Settings with 'yolo settings key=value', i.e. 'yolo settings
 runs_dir=path/to/dir'. For help see
<https://docs.ultralytics.com/quickstart/#ultralytics-settings>.

```
[29]: # Step 2: Load pre-trained YOLOv11 model
model = YOLO('yolov8n.pt')
```

Downloading
<https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt> to
 'yolov8n.pt': 100% 6.2MB 67.7MB/s 0.1s

```
[30]: # Step 3: Function to detect objects using YOLOv11
def detect_objects(frame):
    results = model(frame)
    annotated_frame = results[0].plot()
    return annotated_frame
```

```
[31]: # Step 4: JavaScript to capture video from webcam
def capture_video():
    display(Javascript('''
        async function captureVideo() {
            const video = document.createElement('video');
            const canvas = document.createElement('canvas');
            const context = canvas.getContext('2d');
            document.body.appendChild(video);
            video.style.display = 'none';
            const stream = await navigator.mediaDevices.getUserMedia({video: ↵
↵true});

            video.srcObject = stream;
            await new Promise((resolve) => video.onloadedmetadata = resolve);
            video.play();

            const [track] = stream.getVideoTracks();
            const settings = track.getSettings();
            canvas.width = settings.width;
            canvas.height = settings.height;

            async function stopVideo() {
                stream.getTracks().forEach(track => track.stop());
                video.remove();
```

```

        canvas.remove();
    }

    window.stopVideo = stopVideo;

    while (true) {
        context.drawImage(video, 0, 0, canvas.width, canvas.height);
        const imageData = canvas.toDataURL('image/jpeg', 0.8);
        const response = await google.colab.kernel.
↪invokeFunction('notebook.getFrame', [imageData], {});
        if (response.data.result === 'quit') break;
        await new Promise((resolve) => setTimeout(resolve, 100));
    }

    stream.getTracks().forEach(track => track.stop());
    video.remove();
    canvas.remove();
}
captureVideo();
'''))

```

```

[42]: # Step 5: Python callback to process frames
stop_capture_flag = False # Global flag to signal stopping
last_frame = None # Global variable to store the last processed frame

def get_frame(image_data):
    global stop_capture_flag, last_frame
    try:
        if stop_capture_flag:
            # If the stop flag is set, reset it and signal JS to quit
            stop_capture_flag = False
            return {'result': 'quit'}

        image_data = image_data.split(",")[1]
        image_bytes = base64.b64decode(image_data)
        image = Image.open(BytesIO(image_bytes))
        frame = np.array(image)
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        frame_detected = detect_objects(frame_rgb)
        last_frame = frame_detected # Store the annotated frame
        _, img_encoded = cv2.imencode('.jpg', frame_detected)
        image_widget.value = img_encoded.tobytes()
        return {'result': 'continue'}
    except Exception as e:
        print(f"Error processing frame: {e}")
        return {'result': 'error'}

```

```
[ ]: # Register the Python callback function
from google.colab import output
output.register_callback('notebook.getFrame', get_frame)

# Step 6: Widget to display the video stream
image_widget = widgets.Image(format='jpeg')
display(image_widget)
```

```
[45]: # Step 7: Stop button functionality
def stop_video(change):
    global stop_capture_flag
    stop_capture_flag = True # Set the flag to signal get_frame to quit the JS
    ↪ loop
    eval_js('window.stopVideo()') # Call the JS function to stop the camera and
    ↪ remove HTML elements
    image_widget.value = b'' # Clear the image widget display
```

```
[ ]: # Now call the functions and see the object detection in video

# Start the video capture and object detection
capture_video()

# Create a button to stop the video
stop_button = widgets.Button(description='Stop Video')
stop_button.on_click(stop_video)
display(stop_button)

# Create a button to capture an image
capture_button = widgets.Button(description='Capture Image')
capture_image_widget = widgets.Image(format='jpeg') # Widget to display
    ↪ captured image

def capture_image(change):
    global last_frame
    if last_frame is not None:
        # Convert the last_frame (which is already annotated) to JPEG bytes
        _, img_encoded = cv2.imencode('.jpg', last_frame)
        capture_image_widget.value = img_encoded.tobytes()
        display(capture_image_widget) # Display the captured image
    else:
        print("No frame available to capture.")

capture_button.on_click(capture_image)
display(capture_button)
```

0: 480x640 (no detections), 7.9ms

Speed: 2.1ms preprocess, 7.9ms inference, 0.8ms postprocess per image at shape

(1, 3, 480, 640)